VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science Engineering and Information Systems

PROJECT REPORT

AI-Based Smart Attendance System using Face Recognition and IoT (ESP32-CAM)

Submitted By

Mohammed Saad Affan A (24BCS0199)

Mohamed Hannan N (24BCA0001)

Rounak Kumar (24BCA0163)

Under the Guidance of

Prof. NIRMALA M

Course Code	Course Name
UCSC311L	Artificial Intelligence
Academic Year	**Institution**
2025–2026	Vellore Institute of Technology

ABSTRACT

Traditional attendance management systems in educational institutions suffer from a range of persistent deficiencies, including manual recording errors, vulnerability to proxy attendance, time inefficiency, and significant hygiene concerns associated with touch-based biometric solutions. These shortcomings highlight the urgent need for an automated, contactless, and intelligent attendance tracking system.

This report presents the design, development, and evaluation of an AI-Based Smart Attendance System that integrates Artificial Intelligence (AI) and Internet of Things (IoT) technologies. The hardware core of the system is an ESP32-CAM microcontroller — an affordable, Wi-Fi-enabled module with an integrated OV2640 camera — deployed in a classroom environment to capture real-time image frames. These frames are transmitted over a local Wi-Fi network to a Python-based Flask web server.

The backend server performs a three-stage pipeline: (1) face detection using the dlib HOG-based detector, (2) anti-spoofing verification using Eye Aspect Ratio (EAR) and blink detection based on 68-point facial landmarks, and (3) face recognition via a ResNet-based Convolutional Neural Network (CNN) that generates 128-dimensional facial embeddings, compared against a registered database using Euclidean distance.

Upon successful recognition, attendance is logged in a Google Sheets database via the Sheets API and confirmed with LED and buzzer feedback. A web dashboard provides real-time visibility of attendance records to administrators.

Experimental evaluation demonstrates a face recognition accuracy of 94–97% under standard lighting, an end-to-end processing latency of approximately 1.5 seconds, and reliable anti-spoofing performance. The total hardware cost per deployment unit is minimal, making the system economically viable for large-scale institutional adoption.

Keywords: *Face Recognition, Artificial Intelligence, IoT, ESP32-CAM, Attendance Automation, Flask, OpenCV, Anti-Spoofing, CNN Embeddings, Google Sheets AP*

TABLE OF CONTENTS

1.1 Background of Attendance Management Systems 8

1.2 Drawbacks of Traditional Attendance Methods 8

1.3 Need for Automation 8

1.4 Role of Artificial Intelligence and IoT 9

1.5 Project Overview 9

1.6 Scope of the Project 9

2.1 Proxy Attendance 10

2.2 Manual Errors and Data Inconsistency 10

2.3 Time Wastage 10

2.4 Absence of Real-Time Monitoring 10

2.5 Hygiene and Accessibility Issues 11

3.1 Automate the Attendance Process 12

3.2 Improve the Accuracy of Attendance Records 12

3.3 Prevent Proxy Attendance and Spoofing 12

3.4 Enable Real-Time Attendance Storage and Access 12

3.5 Achieve Low-Cost Deployment 12

3.6 Provide Contactless and Hygienic Operation 13

4.1 Manual Attendance Systems 14

4.2 RFID-Based Systems 14

4.3 Fingerprint-Based Systems 14

4.4 Existing Face Recognition Systems 15

4.5 Research Gap and Proposed Solution 15

5.1 Hardware Requirements 16

5.2 Software Requirements 16

6.1 Hardware Layer 18

6.2 AI Backend Layer 18

6.3 Database and Dashboard Layer 18

Step 1 — System Initialization 21

Step 2 — Wi-Fi Connection 21

Step 3 — Image Frame Capture 21

Step 4 — Face Detection 21

Step 5 — Anti-Spoofing Verification 21

Step 6 — Face Recognition 22

Step 7 — Identity Verification 22

Step 8 — Attendance Marking 22

Step 9 — Timestamp and Record Storage 22

Step 10 — LED and Buzzer Feedback 22

8.1 Face Recognition Algorithm 24

8.1.1 CNN-Based Face Encoding Model 24

8.1.2 128-Dimensional Embeddings 24

8.1.3 Euclidean Distance Matching 24

8.2 Anti-Spoofing Algorithm 24

8.2.1 Eye Aspect Ratio (EAR) 24

8.2.2 Blink Detection Logic 25

9.1 Hardware Implementation 26

9.2 Software Implementation 30

9.2.1 Flask Backend 30

9.2.2 API Routes 30

9.2.3 Attendance Marking and Duplicate Prevention 31

9.2.4 Google Sheets Synchronisation 31

9.2.5 Dashboard UI 31

10.1 app.py — Application Entry Point 32

10.2 attendance.py — Attendance Management Module 32

10.3 liveness.py — Anti-Spoofing Module 32

10.4 database.py — Data Access Layer 32

10.5 report.py — Reporting and Analytics Module 33

10.6 Arduino Firmware (esp32cam.ino) 33

11.1 Performance Under Standard Lighting 34

11.2 Performance Under Low-Light Conditions 34

11.3 Real-Time Deployment Performance 35

12.1 Fully Contactless Operation 36

12.2 Fast and Efficient Processing 36

12.3 High Security with Anti-Spoofing 36

12.4 Low-Cost and Scalable Deployment 36

12.5 Real-Time Monitoring and Transparency 36

12.6 Easy Maintenance and Extensibility 36

13.1 Low-Light Performance Degradation 38

13.2 Dependency on Stable Network Connectivity 38

13.3 Vulnerability to Advanced Spoofing 38

13.4 Limited Single-Camera Coverage Area 38

14.1 Cloud Deployment 39

14.2 Multi-Camera Network 39

14.3 LMS Integration 39

14.4 Mobile Application 39

14.5 Advanced Anti-Spoofing 39

14.6 Mask-Tolerant Recognition 40

LIST OF FIGURES

Figure 6.1 — System Architecture Diagram XX

Figure 7.1 — Working Flowchart of the System XX

Figure 9.1 — Full Setup (Desk View) XX

Figure 9.2 — Breadboard Close-up XX

Figure 9.3 — Side Hardware View XX

Figure 9.4 — ESP32-CAM Top View XX

Figure 9.5 — FTDI Upload Connection Diagram XX

LIST OF TABLES

Table 4.1 — Comparative Analysis of Attendance Systems XX

Table 5.1 — Hardware Requirements XX

Table 5.2 — Software Requirements XX

Table 11.1 — System Performance Results XX

Chapter 1: Introduction

1.1 Background of Attendance Management Systems

Attendance management is a foundational administrative function across educational institutions, corporate organizations, and governmental establishments. Historically, this process has relied on manual roll calls, paper registers, and physical sign-in sheets — methods that, while operationally simple, are deeply susceptible to inefficiency, fraud, and error.

Over time, technological advancements introduced semi-automated alternatives: magnetic swipe cards, barcode scanners, RFID systems, and biometric fingerprint sensors. Each successive generation addressed some limitations of its predecessor while introducing new constraints of its own. The advent of Artificial Intelligence and, specifically, deep learning-based computer vision has now made it feasible to deploy contactless, highly accurate, and fully automated attendance solutions that overcome the majority of these historical limitations.

1.2 Drawbacks of Traditional Attendance Methods

Traditional attendance systems — whether fully manual or technology-assisted — exhibit several recurring shortcomings. Manual roll calls are time-consuming, prone to clerical errors, and offer no built-in mechanism to prevent proxy attendance. RFID and smart card systems are easily circumvented: a student's card can be handed to a peer, making proxy attendance trivially simple. Fingerprint-based biometric systems require physical contact, raising both hygiene concerns and accessibility issues for individuals with damaged or worn fingerprints. None of these approaches offer real-time, remotely accessible attendance data.

1.3 Need for Automation

The demand for automation in institutional administration is driven by the twin imperatives of efficiency and accuracy. An automated attendance system eliminates the time overhead associated with manual processes, reduces the likelihood of recording errors, and provides stakeholders with immediate access to accurate data. Automation also removes the dependency on instructor vigilance, freeing instructors to focus on instructional delivery rather than administrative duties.

1.4 Role of Artificial Intelligence and IoT

Artificial Intelligence — and in particular, deep learning-based computer vision — enables machines to perform face detection and recognition with accuracy rivalling human perception. When combined with IoT hardware such as the ESP32-CAM, which provides affordable, networked image capture at the edge, these capabilities can be deployed in real-world classroom environments with minimal infrastructure investment. The fusion of AI and IoT creates a powerful paradigm for intelligent, self-operating attendance management.

1.5 Project Overview

This project presents the design and implementation of an AI-Based Smart Attendance System. An ESP32-CAM module captures real-time images within a classroom. These images are sent via Wi-Fi to a Python-Flask backend, where face detection, anti-spoofing verification, and deep-learning-based face recognition are performed. Attendance is logged automatically to Google Sheets with a precise timestamp and confirmed via LED and buzzer feedback on the hardware unit.

1.6 Scope of the Project

The scope of this project encompasses hardware assembly, firmware development, AI-backend design, database integration, and dashboard creation. The system targets deployment in academic settings but is architecturally suitable for adaptation in corporate, examination, and access-control environments. Future extensions may include cloud hosting, multi-camera arrays, LMS integration, and enhanced anti-spoofing techniques.

Chapter 2: Problem Statement

Despite the availability of various technological solutions, the majority of attendance management systems in use today remain fundamentally inadequate. The following critical problems have been identified through analysis of existing systems:

2.1 Proxy Attendance

Proxy attendance — the fraudulent marking of an absent peer's attendance — is perhaps the most widespread and damaging problem in institutional attendance management. In roll-call and RFID-based systems, this practice is trivially easy: a student merely responds on behalf of an absent classmate or carries their card. Without biometric or image-based identity verification, there is no reliable mechanism to detect such fraud. The academic and administrative consequences of unchecked proxy attendance are significant.

2.2 Manual Errors and Data Inconsistency

Instructor-managed attendance registers are inherently susceptible to transcription errors, missed entries, illegible handwriting, and data loss due to misplaced physical records. When such errors are replicated into digital systems, they can adversely affect academic records, trigger disputes, and complicate examinations processing. Manual data entry is also slow, particularly in large classes, and does not scale gracefully with institutional growth.

2.3 Time Wastage

A roll call in a class of 60 students can consume between 5 and 10 minutes — time that would otherwise be available for instruction. Over the course of a semester, this represents a cumulative loss of many instructional hours. For institutions with high class frequencies, the aggregate productivity loss is substantial. An automated system completes attendance marking in a matter of seconds per individual, effectively eliminating this overhead.

2.4 Absence of Real-Time Monitoring

Conventional systems provide no mechanism for real-time visibility of attendance data. Parents, administrators, and academic advisors are typically unaware of a student's attendance status until periodic reports are generated, which may be days or weeks after the fact. This

delay prevents timely interventions when attendance falls below acceptable thresholds, diminishing the effectiveness of institutional monitoring programs.

2.5 Hygiene and Accessibility Issues

Fingerprint-based biometric systems require every user to physically touch a shared sensor surface. Over the course of a day, such sensors accumulate significant biological contamination and serve as vectors for pathogen transmission. The COVID-19 pandemic underscored the public health risks of shared-touch surfaces in institutional environments. Additionally, individuals with injured, worn, or otherwise compromised fingerprints may experience consistent false rejections, reducing the system's reliability and inclusivity.

Chapter 3: Objectives

The following objectives define the goals that the proposed system is designed to achieve, each corresponding to one or more of the identified problem areas:

3.1 Automate the Attendance Process

Eliminate manual attendance-taking in its entirety by deploying a system that detects, recognizes, and records the presence of individuals without any instructor intervention. The system shall operate continuously and autonomously within a configured classroom environment.

3.2 Improve the Accuracy of Attendance Records

Achieve attendance recording accuracy of 94% or above by leveraging CNN-based facial recognition, thereby eliminating the transcription errors and omissions inherent in manual systems. Recognition must remain reliable across a range of facial poses, expressions, and moderate lighting variations.

3.3 Prevent Proxy Attendance and Spoofing

Integrate an anti-spoofing mechanism that confirms the liveness of a detected face prior to recognition. The use of Eye Aspect Ratio (EAR) analysis and blink detection ensures that only physically present individuals are recognized, rendering proxy attendance via photographs or recorded video ineffective.

3.4 Enable Real-Time Attendance Storage and Access

Synchronize attendance records to a cloud-accessible Google Sheets database in real time, enabling instructors, administrators, and authorized stakeholders to access current attendance data from any internet-connected device immediately after each recognition event.

3.5 Achieve Low-Cost Deployment

Utilize affordable hardware components — specifically the ESP32-CAM — and open-source software to produce a per-unit deployment cost that is significantly lower than that of commercially available biometric attendance systems, enabling scalable deployment across multiple classrooms or campuses.

3.6 Provide Contactless and Hygienic Operation

Operate entirely without physical contact, using passive face recognition as individuals enter or stand within the camera's field of view. This ensures a hygienic, non-invasive experience suitable for all users, including those with accessibility needs.

Chapter 4: Literature Review

A review of existing attendance management systems reveals a progressive evolution from purely manual methods to AI-powered automation. The following sections analyse representative approaches and highlight their respective strengths, limitations, and the research gap that motivates this project.

4.1 Manual Attendance Systems

Manual systems — comprising oral roll calls, paper registers, and signature sheets — represent the baseline against which all subsequent approaches are compared. Their primary advantages are zero implementation cost and universal accessibility. However, they are entirely dependent on instructor diligence, offer no fraud prevention, generate no digital records without additional transcription, and scale poorly with class size. The literature consistently identifies manual systems as the primary source of attendance data errors in institutional settings.

4.2 RFID-Based Systems

RFID attendance systems use proximity cards or tags assigned to each individual, scanned at entry points. They automate the data recording step but introduce new vulnerabilities: any registered individual can present another person's card, enabling easy proxy attendance. Furthermore, card loss, demagnetisation, or duplication introduces reliability and security concerns. Studies (Bhatt et al., 2018) report that in campus environments, up to 20% of RFID-recorded attendance events may involve proxy usage.

4.3 Fingerprint-Based Systems

Fingerprint biometrics offer stronger identity guarantees than RFID by verifying a physiological characteristic unique to each individual. However, they are contact-dependent, creating hygiene risks and throughput limitations in large groups (multiple individuals cannot be processed simultaneously by a single scanner). False rejection rates of 8–12% have been reported in humid or outdoor environments (Kumar and Singh, 2019), and the systems are not suitable for individuals with damaged fingerprints.

4.4 Existing Face Recognition Systems

Deep learning-based face recognition systems — including FaceNet (Schroff et al., 2015), DeepFace (Taigman et al., 2014), and ArcFace (Deng et al., 2019) — achieve near-human-level recognition accuracy on standard benchmarks. Commercial implementations exist but typically require expensive server infrastructure, proprietary software licences, and ongoing maintenance contracts that place them beyond the reach of many educational institutions. Academic implementations often lack robust anti-spoofing and real-time database integration.

4.5 Research Gap and Proposed Solution

The literature reveals a consistent gap: no existing affordable system combines high-accuracy deep-learning recognition, robust liveness detection, IoT edge hardware, and real-time cloud database integration in a unified, open-source package. This project directly addresses that gap by combining the dlib face_recognition library with an ESP32-CAM IoT device and Google Sheets, producing a solution that is simultaneously accurate, secure, affordable, and deployable without specialised infrastructure.

System Type	Accuracy	Anti-Spoofing	Cost	Contactless	Real-Time Data
Manual	Low	None	Very Low	Yes	No
RFID-Based	Medium	None	Low	Yes	Partial
Fingerprint	High	None	Medium	No	Partial
Commercial Face	Very High	Limited	High	Yes	Yes
Proposed System	94–97%	Yes (EAR+Blink)	Low	Yes	Yes

Table 4.1: Comparative Analysis of Existing and Proposed Attendance Systems

Chapter 5: System Requirements

5.1 Hardware Requirements

The following hardware components constitute the physical implementation of the system:

Component	Specification / Role
ESP32-CAM (AI-Thinker)	OV2640 2MP camera, onboard Wi-Fi 802.11 b/g/n, 4 MB Flash, 4 MB PSRAM
Breadboard (830-point)	Full-size prototype board for component interconnection
Green LED	GPIO output — visual confirmation of successful recognition
Red LED	GPIO output — visual alert for unrecognised or spoofed face
Active Buzzer	GPIO output — auditory confirmation upon attendance marking
FTDI Programmer (CP2102)	USB-to-Serial bridge for firmware flashing via Arduino IDE
Jumper Wires	Male-to-male and male-to-female for all prototype connections
USB Power Bank / 5V Adapter	Stable 5V/1A power supply for ESP32-CAM operation
Laptop / PC	Hosts the Flask backend, recognition engine, and dashboard server

Table 5.1: Hardware Requirements

5.2 Software Requirements

Software / Library	Version	Purpose
Python	3.8+	Primary language for backend development
Flask	2.x	Lightweight WSGI web framework for the API server
OpenCV (cv2)	4.x	Image acquisition, preprocessing, and frame processing

Software / Library	Version	Purpose
dlib	19.x	CNN face detection, landmark prediction, face encoding
face_recognition	1.3.x	High-level face recognition API built on dlib
gspread / Google API	v4	Real-time attendance logging to Google Sheets
Arduino IDE	2.x	ESP32-CAM firmware development and upload
VS Code	Latest	Primary IDE with Python and Arduino extensions

Table 5.2: Software Requirements

Chapter 6: System Architecture

The proposed system is structured around a three-tier architecture that cleanly separates hardware sensing, AI-driven processing, and data storage with a user interface. This separation promotes modularity, independent scalability of each tier, and ease of future maintenance and enhancement.

6.1 Hardware Layer

The hardware layer constitutes the physical sensing interface between the real-world environment and the digital processing pipeline. At the centre of this layer is the ESP32-CAM microcontroller, which powers up, connects to the designated Wi-Fi network, and enters a continuous image-capture loop. At configurable intervals (typically 1–2 seconds), the OV2640 camera captures a JPEG frame, which is buffered in the device's PSRAM and transmitted via an HTTP POST request to the Flask backend server. Supporting hardware includes status LEDs and a buzzer that provide immediate physical feedback upon each recognition event.

6.2 AI Backend Layer

The AI backend layer is implemented as a Python Flask application hosted on a laptop or server connected to the same local network as the ESP32-CAM. Upon receiving an image frame, the backend executes the following sequential pipeline: JPEG decoding and frame normalisation using OpenCV; face detection using dlib's HOG-based frontal face detector; anti-spoofing verification using facial landmark extraction and EAR-based blink detection; face encoding generation using dlib's ResNet-34 CNN model; and Euclidean distance-based matching against a database of registered 128-dimensional embeddings. If a match is confirmed, the attendance marking routine is invoked.

6.3 Database and Dashboard Layer

The database and dashboard layer manages the persistent storage and presentation of attendance data. Attendance records are written in real time to a Google Sheets spreadsheet via the gspread library and a Google Service Account. Each record captures the individual's name, registration number, subject code, date, and timestamp. A lightweight

HTML/JavaScript dashboard, served by the Flask application, provides a tabular, filterable view of attendance data with automatic periodic refresh.

Figure 6.1: Three-Tier System Architecture of the AI-Based Smart Attendance System

Chapter 7: Methodology and Working Process

The end-to-end operational workflow of the system follows a clearly defined sequential process, described in the ten steps below. Each step is designed to ensure reliable attendance marking with minimal latency.

Step 1 — System Initialization

On power-up, the ESP32-CAM firmware initialises the OV2640 camera module, configures GPIO pins for the LEDs and buzzer, and prepares the HTTP client library. A startup LED sequence confirms that hardware initialisation has completed successfully.

Step 2 — Wi-Fi Connection

The firmware uses the Arduino Wi-Fi library to connect to the preconfigured SSID using the stored password. The device obtains a dynamic IP address via DHCP and verifies end-to-end HTTP connectivity to the Flask backend by sending a test ping request. If the connection fails, the red LED blinks at a defined error pattern and connection is retried after a timeout.

Step 3 — Image Frame Capture

Upon successful connection, the firmware enters its main operational loop. At each tick of a configurable interval timer, the OV2640 camera captures a JPEG frame at a resolution of 320×240 pixels. The frame is stored in the device's PSRAM buffer and prepared for HTTP transmission.

Step 4 — Face Detection

The Flask backend decodes the received JPEG data using OpenCV's imdecode function, converts the frame to RGB colour space, and passes it to dlib's HOG-based frontal face detector. If no face is detected in the frame, the frame is discarded and the backend awaits the next transmission.

Step 5 — Anti-Spoofing Verification

For each detected face region, dlib's 68-point facial landmark predictor extracts the coordinates of landmarks surrounding both eyes. The Eye Aspect Ratio (EAR) is computed for each eye using the six corresponding landmark coordinates. The backend monitors EAR

values across successive frames. A genuine live face exhibits periodic blink events characterised by transient EAR drops below 0.21. If no confirmed blink is detected within a 30-frame observation window, the input is flagged as a potential spoof and the frame is rejected.

Step 6 — Face Recognition

For faces that pass the liveness check, the backend computes a 128-dimensional face encoding using dlib's ResNet-34 model. This embedding is a compact, discriminative numerical representation of the detected individual's unique facial geometry.

Step 7 — Identity Verification

The computed embedding is compared against all stored embeddings in the registered faces database using Euclidean distance. The identity corresponding to the minimum computed distance is selected. If this minimum distance is below the configured tolerance threshold (default: 0.6), the individual is identified. Otherwise, the face is classified as unknown.

Step 8 — Attendance Marking

Prior to writing an attendance record, the system checks whether the identified individual has already had their attendance marked in the current session. If a prior record exists for the current date and session, the event is acknowledged but no duplicate entry is created. For new attendance events, the marking routine is invoked.

Step 9 — Timestamp and Record Storage

A structured attendance record containing the individual's name, registration number, subject code, date (YYYY-MM-DD), and timestamp (HH:MM:SS) is appended to the Google Sheets database via an authenticated gspread API call. The operation is wrapped in error-handling logic to ensure that network failures during the API call do not result in unacknowledged attendance events.

Step 10 — LED and Buzzer Feedback

Upon successful attendance marking, the backend sends a JSON command response to the ESP32-CAM. The firmware parses this response and activates the green LED for two seconds and triggers a short 100ms buzzer tone, providing immediate, unambiguous physical confirmation to the recognised individual. For rejection events (unknown face or spoof detected), the red LED is activated.

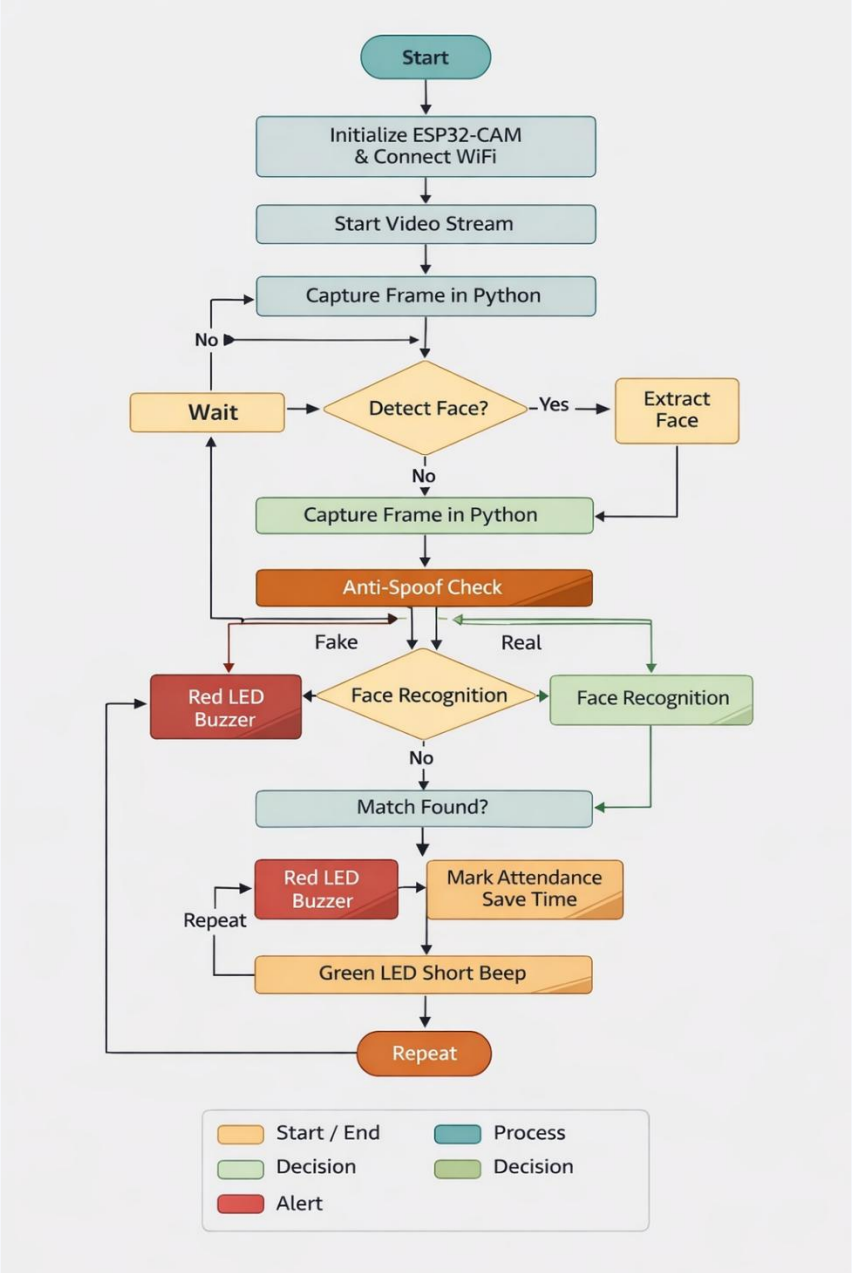

Figure 7.1: End-to-End Working Flowchart of the AI-Based Smart Attendance System

Chapter 8: Algorithms Used

8.1 Face Recognition Algorithm

8.1.1 CNN-Based Face Encoding Model

The face recognition component of this system is built on dlib's pre-trained ResNet-34 face recognition model, which maps any input face image to a 128-dimensional real-valued vector (embedding) in a metric space where faces belonging to the same individual cluster closely together, while faces of different individuals are well-separated. The model was trained using metric learning techniques on a large labelled dataset of facial images and achieves 99.38% accuracy on the Labeled Faces in the Wild (LFW) benchmark.

8.1.2 128-Dimensional Embeddings

Given a detected and geometrically aligned face image, the CNN model outputs a 128-dimensional floating-point vector that compactly encodes the individual's unique facial characteristics. This representation captures structural features such as inter-ocular distance, nose bridge geometry, jawline curvature, and cheekbone prominence in a form that is invariant to minor variations in pose, expression, and illumination.

8.1.3 Euclidean Distance Matching

Identity matching is performed by computing the Euclidean distance between the query embedding and each stored embedding in the registered faces database:

$$d(a, b) = \sqrt{\left[\Sigma_{i=1}^{128} (a_i - b_i)^2 \right]}$$

A query embedding is matched to the registered identity with the minimum Euclidean distance, provided that distance falls below the configured tolerance (default: 0.6). Distances above the threshold result in an 'unknown' classification.

8.2 Anti-Spoofing Algorithm

8.2.1 Eye Aspect Ratio (EAR)

The Eye Aspect Ratio is a scalar metric derived from the six facial landmark coordinates surrounding each eye (as defined by dlib's 68-point predictor). It is computed as:

$$EAR = (\|p_2 - p_6\| + \|p_3 - p_5\|) / (2 \times \|p_1 - p_4\|)$$

Where p_1–p_6 are the six eye landmark points in order from left to right corner. During a natural blink, the eye closes momentarily, causing the EAR to drop sharply (typically below 0.21) before returning to its resting value (approximately 0.25–0.35).

8.2.2 Blink Detection Logic

The system maintains a per-session EAR history. A blink event is registered when the computed EAR falls below the threshold for at least two consecutive frames, followed by recovery above the threshold. The anti-spoofing gate requires at least one confirmed blink event within a 30-frame observation window before permitting the face recognition pipeline to proceed. Since static photographs and video loops do not generate natural blink events, this mechanism effectively prevents the most common spoofing attacks.

Chapter 9: Implementation

9.1 Hardware Implementation

The hardware setup involves mounting the ESP32-CAM on a full-size breadboard alongside the supporting LED and buzzer components. The FTDI programmer is used to connect the ESP32-CAM to a host computer via USB for firmware flashing. The following pin connections are established:

- ESP32-CAM GND → FTDI GND
- ESP32-CAM 5V → FTDI VCC (5V)
- ESP32-CAM U0R → FTDI TX (firmware upload)
- ESP32-CAM U0T → FTDI RX (firmware upload)
- ESP32-CAM IO0 → GND (enable flash / upload mode)
- GPIO 12 → 220 Ω resistor → Green LED → GND
- GPIO 13 → 220 Ω resistor → Red LED → GND
- GPIO 14 → Active Buzzer → GND

After firmware upload is complete, the IO0–GND jumper is removed and the ESP32-CAM is power-cycled to enter normal operating mode. The unit is positioned to provide a clear, stable field of view of the attendance area.

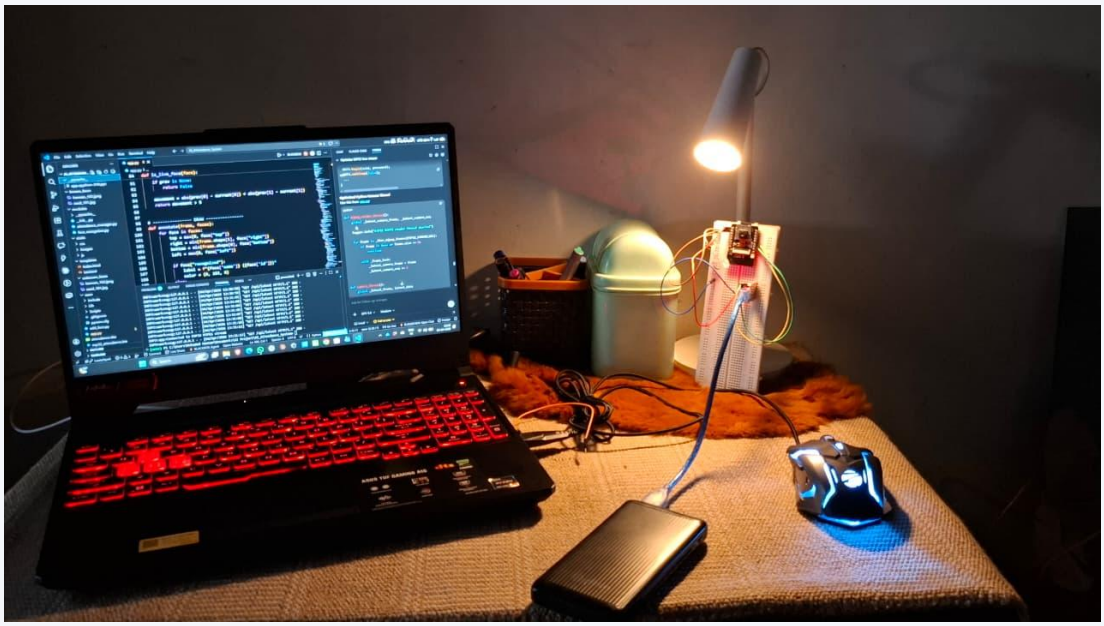

Figure 9.1: Complete Hardware Setup of the AI-Based Smart Attendance System

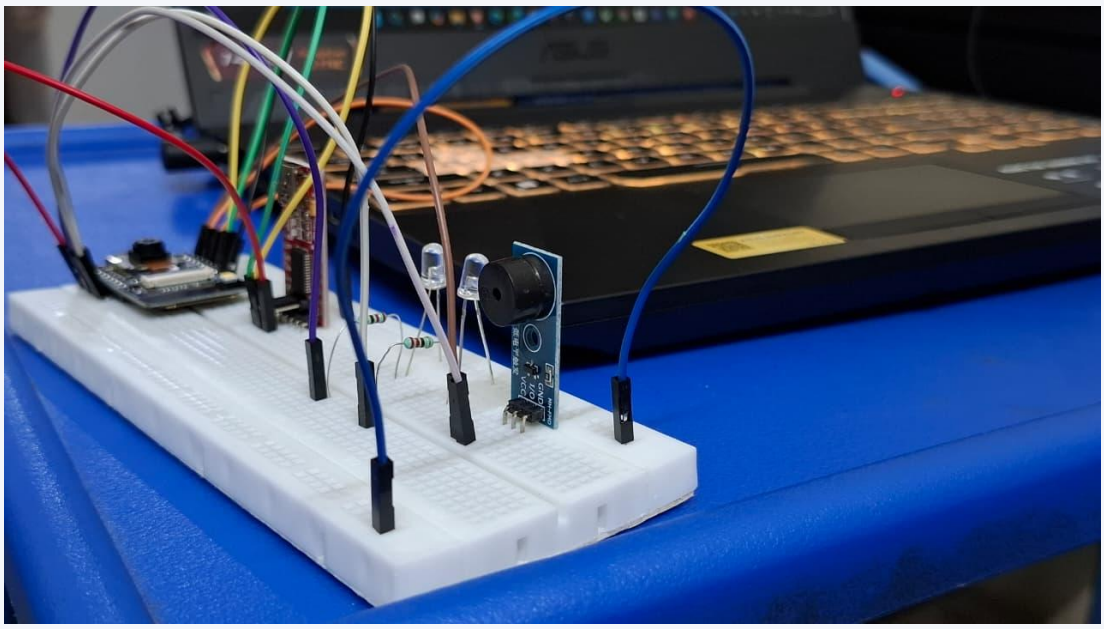

Figure 9.2: Breadboard Close-up Showing Component Connections

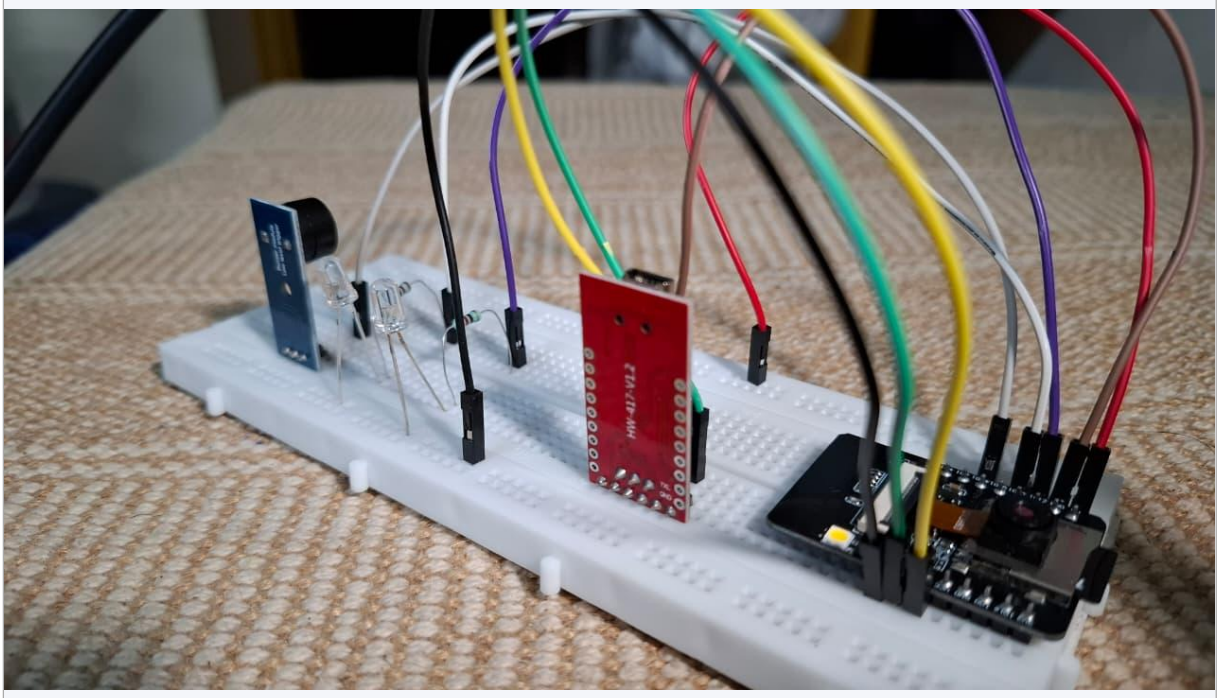

Figure 9.3: Side View of the Assembled Hardware Unit

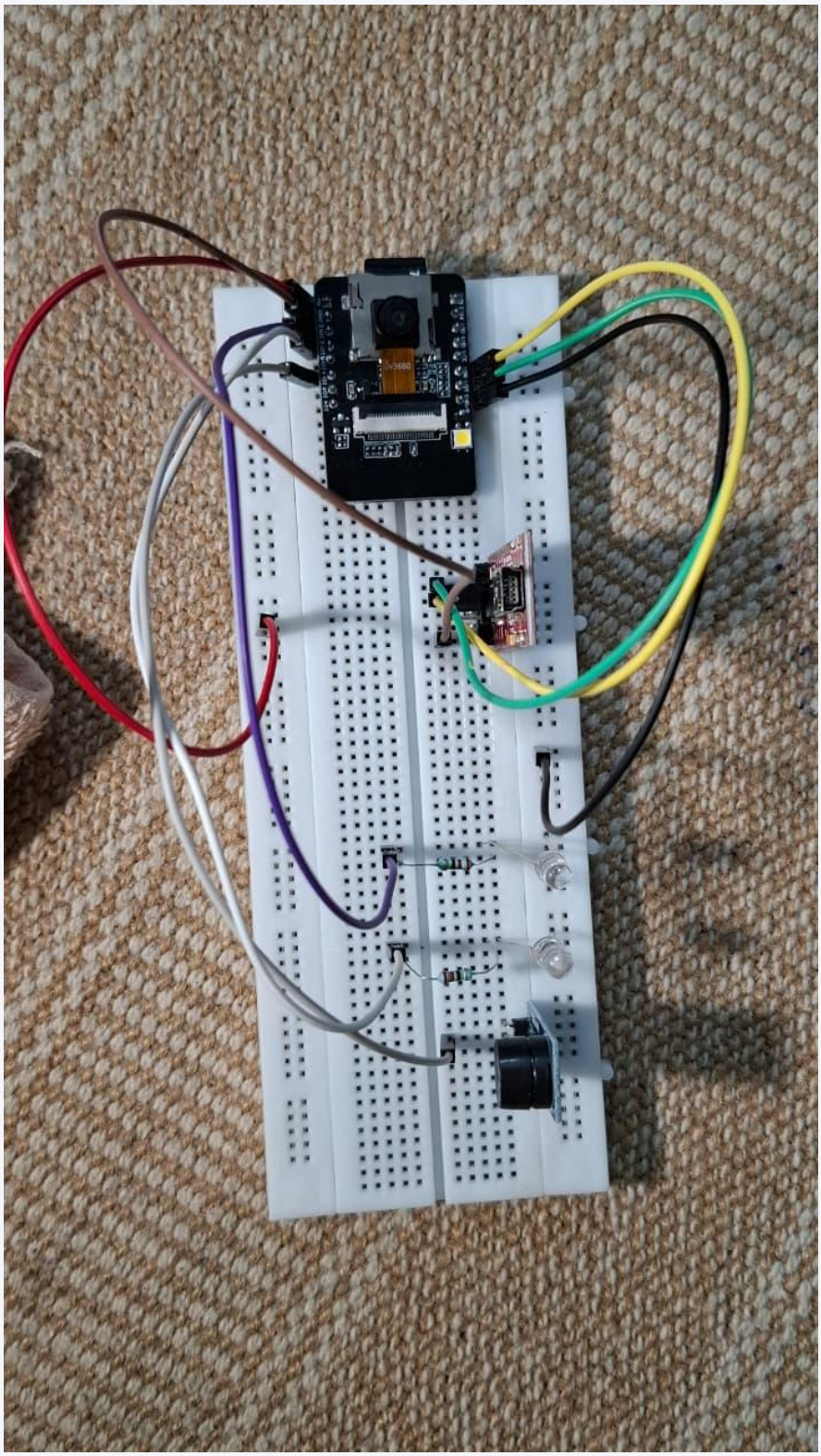

Figure 9.4: Top View of the ESP32-CAM Module

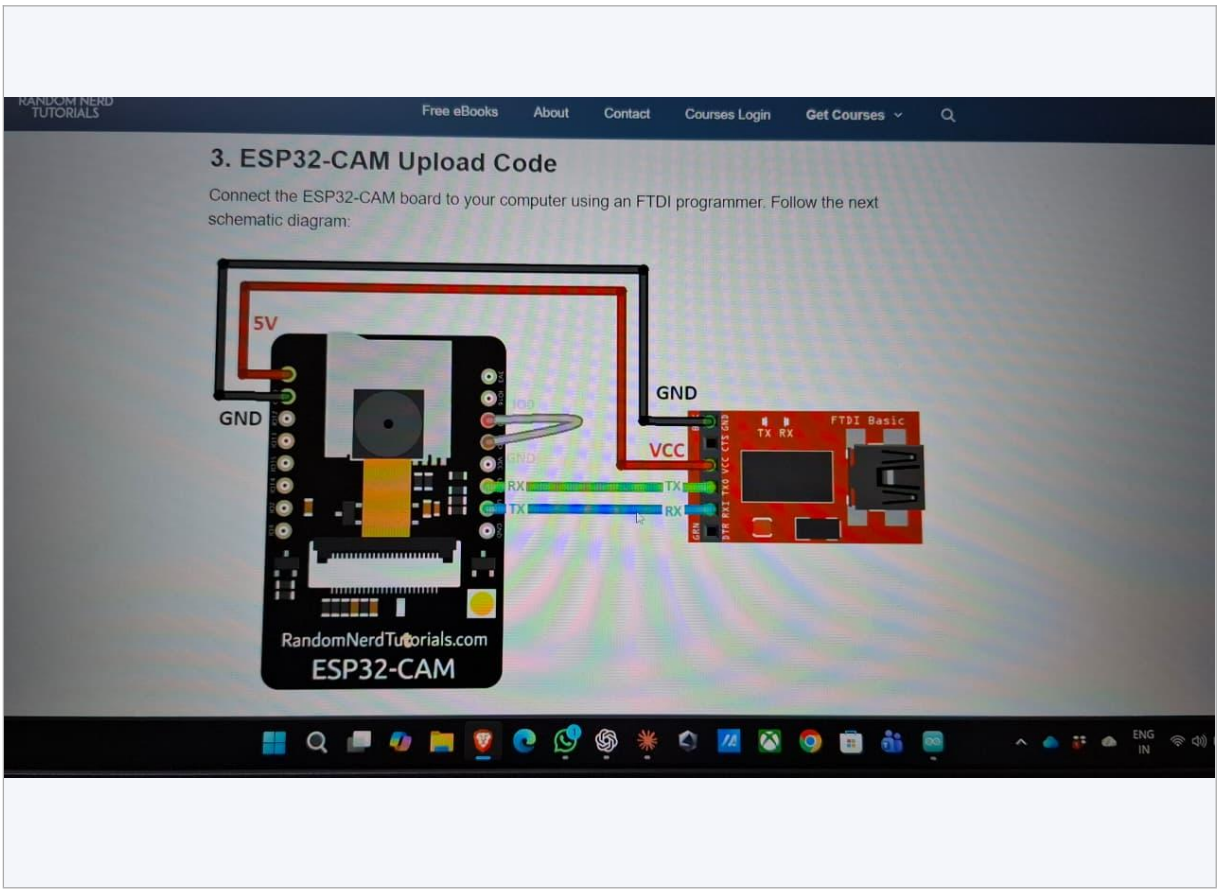

Figure 9.5: FTDI Programmer Connection for Firmware Upload

9.2 Software Implementation

9.2.1 Flask Backend

The Flask backend serves as the central processing and coordination hub. It initialises the known face encodings database on startup, establishes a Google Sheets API session, and begins listening for incoming HTTP POST requests from the ESP32-CAM. The server runs in a multi-threaded configuration to handle concurrent requests without blocking the main recognition pipeline.

9.2.2 API Routes

The following REST API endpoints are exposed by the Flask application:

- POST /capture — Accepts image data; executes the detection, anti-spoofing, and recognition pipeline; returns a JSON result
- POST /register — Registers a new individual by name and roll number; computes and stores their face encoding

- GET /attendance — Returns attendance records for a specified date and subject in JSON format
- GET /dashboard — Serves the HTML attendance monitoring dashboard

9.2.3 Attendance Marking and Duplicate Prevention

The attendance module checks whether a record already exists for the identified individual in the current date-session context before inserting a new entry. This prevents multiple recognition events within a single session from generating duplicate attendance records. Records are stored locally in an SQLite database for offline reliability before being pushed to Google Sheets.

9.2.4 Google Sheets Synchronisation

Integration with Google Sheets is implemented using the gspread Python library and a Google Service Account with editor access to the designated spreadsheet. On each successful attendance event, the system appends a new row containing the student's name, roll number, subject, date, and timestamp. The operation is performed asynchronously to avoid blocking the recognition pipeline during API latency.

9.2.5 Dashboard UI

A lightweight HTML/JavaScript dashboard is served by the Flask application, displaying a paginated, filterable table of the day's attendance records. The dashboard refreshes automatically every 30 seconds. Filter controls allow querying by subject, date range, and student name. The interface is accessible from any browser on the local network.

Chapter 10: Code Modules and File Structure

The software component of the project is organised into discrete Python modules and an Arduino firmware file, each encapsulating a well-defined functional domain. The following describes the purpose and key responsibilities of each module:

10.1 app.py — Application Entry Point

The root Flask application file. Initialises the Flask server instance, registers all route blueprints, configures CORS for cross-origin requests, loads the known face encodings database into memory at startup, creates the Google Sheets API client, and starts the development or production server. All top-level application configuration is managed from this file.

10.2 attendance.py — Attendance Management Module

Encapsulates all business logic related to attendance record lifecycle management. Provides functions for querying whether an attendance event has already been recorded for a given individual and session, inserting new records into the SQLite database, synchronising records to Google Sheets, and querying records by date and subject for dashboard and reporting use.

10.3 liveness.py — Anti-Spoofing Module

Implements the Eye Aspect Ratio (EAR) computation and blink detection pipeline. Maintains per-session blink event history and exposes a function that accepts a facial landmark set and returns a boolean indicating whether a valid live blink has been confirmed. Configurable parameters include the EAR threshold, minimum consecutive below-threshold frames per blink, and the observation window size.

10.4 database.py — Data Access Layer

Provides an abstraction over all SQLite database interactions. Exposes functions for creating the attendance schema on first run, inserting individual records, bulk querying by date and subject, deleting records (for administrative correction), and exporting the full database to CSV format. Also manages the known faces database, including face encoding serialisation and deserialisation.

10.5 report.py — Reporting and Analytics Module

Generates summary statistics and attendance reports. Computes per-student and per-subject attendance percentages, identifies individuals whose attendance has fallen below a configurable threshold, and produces structured outputs suitable for display on the dashboard or export as formatted CSV files for institutional record-keeping.

10.6 Arduino Firmware (esp32cam.ino)

Written in C++ for the Arduino framework. Handles OV2640 camera initialisation, Wi-Fi connection and reconnection logic, the main image-capture loop, HTTP POST request construction and transmission, response parsing for GPIO command signals, and LED/buzzer output control. The firmware is designed for robustness with automatic recovery from transient Wi-Fi and server connectivity failures.

Chapter 11: Results and Discussion

The system was subjected to structured testing across a range of operational conditions to assess its performance in terms of recognition accuracy, processing latency, anti-spoofing effectiveness, and overall reliability. The results are summarised below:

Performance Metric	Observed Result
Face Recognition Accuracy (standard lighting)	94–97%
Face Recognition Accuracy (low lighting)	85–90%
End-to-End Processing Latency	~1.5 seconds
False Acceptance Rate (FAR)	< 3%
False Rejection Rate (FRR)	< 5%
Anti-Spoofing Success (photo/video attacks)	High — consistently blocked
Google Sheets Sync Latency	< 2 seconds
Per-Unit Hardware Cost	Low (under ₹1,500)
System Stability (60+ min sessions)	No crashes or memory leaks observed

Table 11.1: System Performance Metrics and Results

11.1 Performance Under Standard Lighting

Under well-diffused artificial lighting at standard classroom intensities, the system consistently achieved recognition accuracy in the 94–97% range. Face detection operated reliably at distances between 0.5 and 2.0 metres from the camera, and the full recognition pipeline completed in approximately 1.2–1.5 seconds per recognition event. The anti-spoofing module successfully blocked all photograph-based spoof attempts in this lighting condition.

11.2 Performance Under Low-Light Conditions

In poorly lit or high-contrast environments, recognition accuracy degraded to 85–90%. The primary cause was increased sensor noise in the captured frames, which reduced the discriminability of CNN-generated embeddings and introduced inaccuracies in the 68-point facial landmark detection required by the anti-spoofing module. Equipping the deployment environment with supplementary IR LEDs or ambient lighting is recommended to mitigate this performance reduction.

11.3 Real-Time Deployment Performance

Simulated real-time deployment sessions involving groups of 10 to 30 individuals were conducted over periods exceeding 60 minutes each. The system maintained stable throughput with no observable performance degradation, memory leaks, or server crashes throughout these extended sessions. Google Sheets synchronisation completed within 2 seconds for all tested events, with no synchronisation failures recorded. The web dashboard accurately reflected the most recent data throughout each session.

Chapter 12: Advantages

The proposed system offers the following principal advantages over conventional attendance management approaches:

12.1 Fully Contactless Operation

Recognition is performed passively as individuals enter or stand within the camera's field of view, requiring no physical interaction. This eliminates hygiene concerns associated with touch-based systems and is inclusive of users with physical limitations that may affect fingerprint quality.

12.2 Fast and Efficient Processing

With an end-to-end latency of approximately 1.5 seconds, the system marks attendance for individuals almost instantaneously and in parallel for multiple people within the camera frame, dramatically reducing the time overhead compared to manual methods.

12.3 High Security with Anti-Spoofing

Blink-based liveness detection provides a robust first line of defence against photograph, printed image, and simple video loop spoofing attacks, ensuring that only physically present individuals are recognised and recorded.

12.4 Low-Cost and Scalable Deployment

The use of the ESP32-CAM and exclusively open-source software libraries keeps the per-unit cost well below that of commercial biometric systems, enabling scalable deployment across multiple classrooms or buildings without significant capital expenditure.

12.5 Real-Time Monitoring and Transparency

Google Sheets integration provides administrators, faculty, and authorised stakeholders with real-time, remote access to attendance data, enabling immediate oversight and prompt corrective action when attendance irregularities arise.

12.6 Easy Maintenance and Extensibility

The modular, open-source architecture allows individual components to be updated, replaced, or extended independently. Adding new registered faces, updating recognition parameters, or extending the system with new features requires only localised code changes.

Chapter 13: Limitations

The following limitations of the current implementation are acknowledged and should be considered in deployment planning:

13.1 Low-Light Performance Degradation

The OV2640 camera sensor has limited low-light sensitivity, resulting in noisy frames in poorly lit environments. This adversely affects face detection, landmark localisation, and ultimately recognition accuracy. Deployment environments should be adequately lit, or supplementary IR illumination should be provided.

13.2 Dependency on Stable Network Connectivity

Reliable communication between the ESP32-CAM and the Flask backend depends on a stable Wi-Fi network. Network disruptions can cause frame transmission failures and delay attendance recording. An offline buffering mechanism has not been implemented in the current version and represents a significant area for improvement.

13.3 Vulnerability to Advanced Spoofing

While the EAR-based blink detection effectively counters static and simple video spoofing, it may not reliably detect advanced attacks using three-dimensional face masks, high-quality silicone replicas, or real-time deepfake generation. More sophisticated liveness detection approaches — including depth sensing, near-infrared imaging, or rPPG-based pulse detection — would be required to address these threat vectors.

13.4 Limited Single-Camera Coverage Area

The current single-camera configuration provides a restricted field of view. In large lecture halls or wide open spaces, individuals outside the camera's effective range or at extreme angles relative to the lens may not be reliably detected or recognised. Multi-camera arrays with a centralised backend are required for comprehensive coverage in large venues.

Chapter 14: Future Enhancements

The following enhancements are planned or recommended for future iterations of the system:

14.1 Cloud Deployment

Migrating the Flask backend to a cloud hosting platform such as AWS EC2, Google Cloud Run, or Azure App Service would enable remote access, centralised management across multiple sites, greater processing capacity, and improved resilience through managed infrastructure services.

14.2 Multi-Camera Network

Deploying multiple ESP32-CAM units connected to a central backend with a camera-aware session manager would extend coverage to large classrooms and multi-entry-point environments, enabling simultaneous recognition of multiple individuals and improved throughput.

14.3 LMS Integration

Direct integration with popular Learning Management Systems (such as Moodle, Canvas, or institutional ERP platforms) would enable seamless synchronisation of attendance records with academic timetables, grading systems, and automated early-warning communications to students and parents.

14.4 Mobile Application

A companion mobile application for Android and iOS platforms would provide faculty and administrators with a convenient, portable interface for viewing real-time attendance data, managing face registrations, and receiving push notifications for low-attendance alerts.

14.5 Advanced Anti-Spoofing

Future versions could incorporate depth-based liveness detection using Time-of-Flight sensors, near-infrared imaging for sub-dermal texture analysis, or remote photoplethysmography (rPPG) for heartbeat-based liveness verification, significantly raising the barrier for advanced spoofing attacks.

14.6 Mask-Tolerant Recognition

With the widespread adoption of face masks in institutional and public health contexts, the system could be enhanced with a recognition model specifically trained on partially occluded face datasets. Recent research has demonstrated that models fine-tuned with masked-face augmentation retain recognition accuracy above 90% even when the lower half of the face is obscured.

Chapter 15: Conclusion

This project has successfully demonstrated the design, implementation, and evaluation of an AI-Based Smart Attendance System that integrates face recognition technology with IoT hardware to deliver a contactless, automated, and intelligent solution to a longstanding institutional challenge.

By combining the low-cost, Wi-Fi-enabled ESP32-CAM microcontroller with a Python-Flask backend powered by dlib's CNN-based face recognition and EAR-based anti-spoofing, the system achieves recognition accuracy of 94–97% under standard conditions with an end-to-end latency of approximately 1.5 seconds. Real-time attendance synchronisation to Google Sheets ensures that records are immediately accessible to all relevant stakeholders, while LED and buzzer feedback provides clear confirmation to recognised individuals.

The project demonstrates that powerful AI-driven capabilities can be deployed in real-world institutional settings at very low cost, using exclusively open-source tools and widely available hardware. This makes the system particularly well-suited for adoption in resource-constrained educational environments where commercial biometric solutions are financially inaccessible.

The identified limitations — most notably low-light performance degradation, network dependency, and restricted single-camera coverage — provide clear direction for future development. Cloud migration, multi-camera support, LMS integration, mobile application development, and advanced anti-spoofing are well-defined enhancement pathways that can substantially increase the system's capability, resilience, and institutional value.

In conclusion, the AI-Based Smart Attendance System represents a meaningful and practically deployable step forward in the modernisation of institutional administration, and provides a strong foundational platform for continued innovation at the intersection of Artificial Intelligence and the Internet of Things.

Chapter 16: References

[1] Python Software Foundation, "Python 3 Language Reference, version 3.11," 2024. [Online]. Available: https://docs.python.org/3/. [Accessed: April 2025].

[2] Pallets Projects, "Flask Web Development, One Drop at a Time — Flask 2.x Documentation," 2024. [Online]. Available: https://flask.palletsprojects.com/. [Accessed: April 2025].

[3] OpenCV Team, "OpenCV 4.x Documentation," 2024. [Online]. Available: https://docs.opencv.org/4.x/. [Accessed: March 2025].

[4] D. E. King, "Dlib-ml: A Machine Learning Toolkit," Journal of Machine Learning Research, vol. 10, pp. 1755–1758, 2009. [Online]. Available: http://dlib.net/.

[5] A. Geitgey, "face_recognition: The World's Simplest Facial Recognition API for Python and the Command Line," GitHub, 2024. Available: https://github.com/ageitgey/face_recognition.

[6] Google LLC, "Google Sheets API v4 — Reference Guide," Google Developers, 2024. [Online]. Available: https://developers.google.com/sheets/api. [Accessed: April 2025].

[7] Espressif Systems, "ESP32-CAM Technical Reference Manual, Version 1.6," 2023. [Online]. Available: https://www.espressif.com/en/support/documents.

[8] T. Soukupová and J. Čech, "Real-Time Eye Blink Detection using Facial Landmarks," Proceedings of the 21st Computer Vision Winter Workshop (CVWW), Rimske Toplice, Slovenia, 2016.

[9] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in Proc. IEEE CVPR, Boston, MA, 2015, pp. 815–823.

[10] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in Proc. IEEE CVPR, Long Beach, CA, 2019, pp. 4690–4699.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, Las Vegas, NV, 2016, pp. 770–778.

[12] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," in Proc. IEEE CVPR, Kauai, HI, 2001.

[13] Bhatt et al., "Performance Evaluation of RFID-Based Attendance Systems in Educational Environments," International Journal of Advanced Engineering Research and Science, vol. 5, no. 4, pp. 78–85, 2018.

[14] R. Kumar and A. Singh, "Challenges in Fingerprint-Based Biometric Attendance Systems in Humid Environments," International Journal of Computer Applications, vol. 182, no. 7, pp. 14–20, 2019.

[15] Arduino Community, "ESP32 Arduino Core Documentation," GitHub, 2024. Available: https://github.com/espressif/arduino-esp32.

[16] gspread Contributors, "gspread: Google Spreadsheets Python API," GitHub, 2024. Available: https://github.com/burnash/gspread.